Guion del video de defensa

Duración objetivo: 11 minutos (rango permitido: 8 a 12). **Formato**: 7 bloques grabados por separado y unidos al final en Clipchamp. **Change del ciclo OPSX grabado**: `C-05` `reprogramar-turno`, hecho desde cero frente a cámara.

Enfoque: el protagonista es el método, no el producto

Lo que se evalúa es que entienda **cómo funciona Active Stack y la metodología OpenSpec**. El sistema de turnos es solo el ejemplo con el que lo apliqué. Por eso cada bloque responde las mismas cuatro preguntas:

1. **¿Para qué sirve esta fase?** Qué problema resuelve.
2. **¿Qué pasaría si me la salteo?** Esto demuestra que se entiende la fase.
3. **¿Qué produce y quién lo usa después?** Cómo se encadena con la fase siguiente.
4. **¿Dónde intervine yo?** El punto de control humano: Continuar, Ajustar o Parar.

El contenido de cada archivo se muestra **solo como evidencia**, unos segundos, con **un** ejemplo concreto. No se leen archivos ni se explican reglas del dominio en detalle.

> **Trabajo individual**: narro todo con mi voz real, sin voz sintética. El texto sugerido es un ayudamemoria: **no lo leas**, decilo con tus palabras. Lo que está entre `[corchetes]` se completa cuando ocurra.

Mapa mental del stack (para tenerlo claro antes de grabar)

Pieza	Qué es	Para qué sirve
SDD (Spec-Driven Development)	La metodología.	La especificación aprobada es la fuente de verdad y el código es su consecuencia. Primero se acuerda *qué* tiene que pasar y después se programa.
OpenSpec (`openspec` CLI)	La herramienta que implementa SDD.	Guarda el estado: `openspec/specs/` describe lo que el sistema hace **hoy** y `openspec/changes/` lo que se está **cambiando**. `openspec status` dice qué artefactos faltan.
Ciclo OPSX	Las fases de un change: explore → propose → apply → archive.	Cada cambio pasa por pensar, acordar, implementar y consolidar.
Active Stack	El stack completo alrededor del agente.	Une la fundación del proyecto, OpenSpec, las skills, la memoria y las reglas de gobierno, para que el agente trabaje con contexto y límites.
Agente (Claude Code) + orquestador	El que ejecuta.	El orquestador coordina y delega cada fase en subagentes con contexto limpio.

Pieza	Qué es	Para qué sirve
Skills	Paquetes de instrucciones especializadas.	El agente carga las buenas prácticas de la tecnología (FastAPI, PostgreSQL, TDD) en vez de improvisarlas.
Engram	La memoria persistente.	Las decisiones sobreviven entre sesiones: un chat nuevo no arranca de cero.
Governance y TDD estricto	Las reglas de autonomía y de calidad.	Según el riesgo, el agente pide permiso o avanza solo. Ningún código se escribe sin un test que lo pida.

Antes de grabar (checklist)

- ☐ Docker Desktop levantado (para los tests y la demo).
- ☐ `openspec list` vacío y `git status` limpio: el ciclo arranca desde cero.
- ☐ Terminal con letra grande (Ctrl + rueda) y tema legible.
- ☐ El archivo `.env` cerrado y nunca abierto en el explorador: **no debe verse ninguna clave**.
- ☐ Solo datos ficticios en pantalla (los del seed).
- ☐ Micrófono probado.
- ☐ Cámara encendida para los bloques 1 y 7.

Qué se graba en vivo y qué no

Bloque	Cuándo se graba
1, 2 y 7	En cualquier momento: muestran lo que ya existe.
3, 4, 5 y 6	**Durante el chat de C-05.** Grabá la sesión entera de corrido y después recortá.

Arrancá a grabar **antes** del primer mensaje del chat nuevo: así el bloque 6 (Engram) queda registrado aunque en el video vaya después del 5.

Guion de la sesión en vivo de C-05

Regla de control: después de cada paso, el agente se detiene y espera tu OK. No se commitea nada sin que lo pidas, y se hace un commit por etapa.

#	Qué escribís	Qué tiene que pasar	Bloque
1	*Retomemos el proyecto. Recuperá el contexto de Engram y el estado de OpenSpec, y arranquemos con C-05 reprogramar-turno. Frená después de cada paso y esperá mi OK.*	Consulta Engram, corre `openspec list` y resume lo decidido y lo que C-05 **no** lleva.	6
2	`/opsx:explore reprogramar-turno`	Plantea las dudas que la KB no responde. Las decidís en voz alta.	3
3	`/opsx:propose reprogramar-turno`	Crea proposal, design, specs y tasks. **Se frena.**	3

#	Qué escribís	Qué tiene que pasar	Bloque
4	Revisás el spec (checklist del bloque 3)	Encontrás algo real para corregir.	3
5	`/opsx:update reprogramar-turno` + la corrección	Spec ajustado; mostrás `git diff`. Commit `docs(c-05): propose…`.	3
6	`/opsx:apply reprogramar-turno`	TDD estricto, un commit por tarea.	4
7	*Corré la suite completa*	`pytest` todo en verde.	4
8	Demo en Swagger	La funcionalidad andando.	4
9	`/opsx:archive reprogramar-turno`	Fusiona el delta en `openspec/specs/` y mueve el change a `archive/`.	5

Bloque 1 — Por qué empezar por el Discovery (1:15)

Toma: cámara (0:15) → pantalla (1:00).

Pregunta	Respuesta (con tus palabras)
¿Para qué sirve?	Decidir **qué** construir antes de pensar en el **cómo**: investigar el mercado, encontrar un vacío real y recortar un MVP.
¿Si me la salteo?	El agente construye lo primero que se le ocurre, o algo que ya existe.
¿Qué produce y quién lo usa?	`docs/discovery/` (informe y checklist). Lo consume la etapa siguiente, `kb-creator`, para armar la base de conocimiento.
¿Dónde intervine?	Usé **Ajustar** para recortar el MVP y verifiqué fuentes a mano.

Mostrar (evidencia, pocos segundos cada una): el resumen ejecutivo, la matriz de puntuación y la sección D.4 del informe.

Texto sugerido:

> Soy [nombre]. Para este trabajo apliqué Active Stack y OpenSpec a un sistema de turnos para consultorios odontológicos, pero lo que quiero mostrar es el método.
>
> La primera fase es el Discovery. Sirve para decidir qué construir antes de escribir una línea: el agente investiga el mercado con fuentes y fechas, y propone un MVP.
>
> Acá aprendí la primera lección del método: el agente propone y yo decido. Propuso un MVP enorme y lo recorté con Ajustar. Además verifiqué fuentes a mano y encontré errores: el agente, cuando no encuentra un dato, tiende a completarlo. Por eso la verificación humana es parte del proceso, no un extra.

Bloque 2 — La fundación de Active Stack: para qué sirve cada etapa (2:15)

Toma: pantalla. Mostrá cada carpeta **solo como resultado** de la etapa que la generó.

`/active-orchestrator` encadena la fundación en este orden. Cada etapa consume lo que produjo la anterior, y en todas tengo **Continuar, Ajustar o Parar**:

Etapa	¿Para qué sirve?	¿Qué produce?	¿Quién lo usa después?
`openspec init`	Prepara el proyecto para trabajar con specs.	`openspec/` (con `specs/`, `changes/` y `config.yaml`)	Todo el ciclo OPSX.
`discovery-research`	Decidir qué construir (bloque 1).	`docs/discovery/`	`kb-creator`
`kb-creator`	Convertir el Discovery en una **fuente de verdad del dominio** que el agente lee antes de implementar: así no tiene que adivinar reglas.	`knowledge-base/`: 10 archivos canónicos	`roadmap-generator`, y cada change al hacer propose
`roadmap-generator`	Partir el MVP en **changes chicos y ordenados por dependencias**, para no pedir "hacé todo el sistema".	`CHANGES.md` (árbol, camino crítico, governance de cada change)	Me dice qué change sigue y qué leer antes
`find-skill`	Buscar e instalar skills de la tecnología elegida, para que el agente aplique buenas prácticas y no improvise.	Skills instaladas y su justificación en `docs/fundacion/`	`skill-registry`
`skill-registry`	Indexar qué skill corresponde a cada tipo de tarea.	`.atl/skill-registry.md`	El orquestador, al armar el brief de cada subagente
`agent-instruction`	Darle al agente las **reglas del proyecto** desde el primer mensaje.	`AGENTS.md` / `CLAUDE.md`: stack, skills por rol y reglas duras	Cualquier agente que entra al repo

Mostrar (rápido, como evidencia): el árbol del repo; `knowledge-base/` (solo la lista de archivos); `CHANGES.md` (el árbol de dependencias); `.atl/skill-registry.md`; la sección "Reglas Duras" de `AGENTS.md`; `git log --oneline`.

Texto sugerido:

> La fundación la arma `/active-orchestrator`, que encadena siete etapas. Lo importante es que cada una produce algo que consume la siguiente: el Discovery alimenta la base de conocimiento, la base alimenta el roadmap, y así.
>
> La base de conocimiento existe para que el agente no invente reglas del dominio: antes de implementar, la lee. Si una regla no está, la pregunta, no la supone. Y ahí también usé Ajustar, porque el agente había dado por hecho supuestos que no eran ciertos.

> El roadmap parte el MVP en changes chicos con sus dependencias. Esto es clave en SDD: cada change tiene que ser lo bastante chico como para especificarlo y verificarlo completo.
>
> Las skills le dan al agente conocimiento experto de cada tecnología, y el registry le dice cuál cargar en cada tarea.
>
> `AGENTS.md` es lo primero que lee cualquier agente: tiene reglas duras que no se negocian, por ejemplo que el agente nunca commitea sin que yo se lo pida.
>
> Cada etapa quedó en su propio commit, así que el historial cuenta el proceso.

Bloque 3 — El ciclo OPSX: Explore, Propose y Ajustar (2:30) ★ el más importante

Toma: pantalla, en vivo (pasos 2 a 5).

Explore

Pregunta	Respuesta
¿Para qué sirve?	Pensar **antes** de comprometerse: el agente investiga y saca a la luz las ambigüedades, sin escribir código ni artefactos.
¿Si me la salteo?	Las dudas se resuelven en silencio, y es el agente el que decide por mí.
¿Dónde intervine?	Respondí las dudas que planteó. [Ejemplo: si al reprogramar se puede cambiar la prestación.]

Propose

Pregunta	Respuesta
¿Para qué sirve?	Crear el change con **todos** sus artefactos, que forman un grafo: `proposal.md` (qué y por qué) → `specs/` (qué debe cumplirse, en escenarios dado/cuando/entonces) y `design.md` (cómo) → `tasks.md` (pasos). `openspec status` muestra qué está listo.
¿Por qué *delta* specs?	El spec del change no reescribe todo: describe solo lo que se **agrega o modifica** respecto de `openspec/specs/`.
¿Si me la salteo?	No hay contra qué verificar el código: "terminado" pasa a significar "compila".
¿Qué produce y quién lo usa?	La carpeta `openspec/changes/reprogramar-turno/`. Apply la usa como contrato.

Ajustar (revisión humana)

Pregunta	Respuesta
¿Para qué sirve?	Es el **punto de control** del método: el error se corrige en un documento, que cuesta minutos, y no en código.
¿Si me la salteo?	Apply implementa fielmente un spec equivocado: todos los tests pasan y el sistema hace otra cosa.

Mostrar: los comandos, la estructura de la carpeta del change (no el contenido completo), **un** escenario dado/cuando/entonces, el momento en que encontrás el error y el `git diff` del ajuste.

Texto sugerido:

> Elegí C-05 porque es una sola funcionalidad, depende solo de lo que ya está hecho y tiene reglas que se pueden verificar con tests.
>
> Explore sirve para pensar antes de comprometerse. El agente detectó que la base de conocimiento no decía [duda], y la decidí yo.
>
> Propose crea el change con todos sus artefactos, que dependen entre sí: primero el porqué, después qué debe cumplirse y cómo, y al final las tareas. El spec es un delta: solo describe lo que cambia respecto del sistema actual.
>
> Y acá está el punto de control. No lo aprobé a la primera: encontré que [qué estaba mal]. Si hubiera seguido, Apply lo habría implementado tal cual y los tests lo habrían validado, porque los tests salen del spec. Por eso la revisión humana va acá, y no al final.

Checklist de revisión (para que el Ajustar sea real): - ¿Se coló alcance de otro change? Las reglas de horarios y bloqueos son de C-10 y C-11; auditoría y recordatorio, de C-07 y C-12; autenticación, de C-06. - ¿Faltan escenarios de borde? Por ejemplo: solaparse consigo mismo es válido; un error deja el original intacto; un turno cancelado no se reprograma; un turno inexistente da 404; concurrencia. - ¿Hay algo inventado que no figura en la KB?

Bloque 4 — Apply y Verify: el spec se convierte en código (2:00)

Toma: pantalla, en vivo (pasos 6 a 8).

Pregunta	Respuesta
¿Para qué sirve Apply?	Implementar **solo** lo que dice el spec aprobado, tarea por tarea. Si falta algo, no se improvisa: se vuelve a Propose.
¿Cómo trabaja el agente?	El orquestador delega en un subagente, que carga las skills del registry y aplica **TDD estricto**: test en rojo → código mínimo → verde → refactor. Por la governance, en un dominio de riesgo MEDIO avanza con checkpoints y me consulta las decisiones no obvias.
¿Para qué sirve Verify?	Comprobar que **cada escenario del spec tiene un test que pasa**. Así se cierra el círculo: escenario → test → código.

Pregunta	Respuesta
¿Si me la salteo?	No hay evidencia de que el código cumple el spec. Nada se archiva en rojo.

Mostrar: 1. `tasks.md` con las tareas marcadas. 2. **Un** escenario del spec al lado de su test (la trazabilidad). 3. `pytest` todo en verde, incluidos los tests de changes anteriores. 4. Demo rápida en Swagger (`http://localhost:8000/docs`): reprogramar a un hueco libre da 200; a uno ocupado, 409.

Preparación de la demo (fuera de cámara): `docker compose up -d db api`, migraciones y seed (ver `README.md`); anotá IDs del seed y usá fechas futuras.

Texto sugerido:

> Apply toma el spec aprobado como contrato. El agente no decide qué hacer, solo cómo hacerlo, y lo hace con TDD: primero un test que falla, después el código mínimo para que pase.
>
> Verify cierra el círculo: este escenario del spec tiene este test, y el test pasa. La suite completa está en verde, incluidos los tests de los changes anteriores, así que el cambio nuevo no rompió nada.
>
> Y para que se vea funcionando: reprogramo a un horario libre y anda; a uno ocupado lo rechaza.

Bloque 5 — Archive: el spec vigente se actualiza (0:45)

Toma: pantalla, en vivo (paso 9).

Pregunta	Respuesta
¿Para qué sirve?	Consolidar: el delta spec se **fusiona** en `openspec/specs/`, que vuelve a describir lo que el sistema hace hoy, y el change pasa a `archive/` con fecha.
¿Si me la salteo?	`openspec/specs/` queda desactualizado y el próximo change se propone sobre una descripción falsa del sistema.
¿Qué produce y quién lo usa?	Specs vigentes actualizados, que son la base del próximo propose, y el historial en `archive/`: por qué se hizo cada cosa.

Mostrar: el comando; `openspec list` vacío; la carpeta en `archive/`; el spec vigente con los requisitos nuevos; C-05 en `[x]` en `CHANGES.md`.

Texto sugerido:

> Archive cierra el ciclo. Sirve para que la especificación del sistema siga siendo verdad: los requisitos de reprogramar pasan al spec vigente, y el change queda archivado con su propuesta, diseño y tareas. Si dentro de un año alguien pregunta por qué funciona así, la respuesta está ahí.

Bloque 6 — Engram: memoria entre sesiones (1:00)

Toma: pantalla. Es el arranque del chat nuevo (paso 1).

Pregunta	Respuesta
¿Para qué sirve?	Que las decisiones sobrevivan entre sesiones y compactaciones del contexto. El agente guarda lo importante y lo recupera al empezar.
¿Si no estuviera?	Cada chat arranca de cero: hay que reexplicar todo, o el agente vuelve a tomar decisiones que ya estaban tomadas, distintas.
¿Cómo se complementa con OpenSpec?	OpenSpec guarda el **estado de los artefactos** (qué está especificado y hecho); Engram guarda **el porqué y lo aprendido** en el camino.

Mostrar: el primer mensaje, el agente consultando Engram y `openspec list`, y su resumen.

Texto sugerido:

> Engram es la memoria persistente del stack. Abrí un chat nuevo, sin contexto, y el agente recuperó solo las decisiones anteriores: sabe qué queda fuera de C-05 sin que yo se lo explique. OpenSpec le dice en qué estado está el proyecto, y Engram, por qué se decidió cada cosa.

Bloque 7 — Conclusiones (1:00)

Toma: cámara.

Texto sugerido (con tus palabras):

> La idea central de SDD es que primero se acuerda qué tiene que pasar y después se programa. Active Stack le da al agente todo lo que necesita para respetarlo: una base de conocimiento, un roadmap, skills, reglas y memoria.
>
> Lo que más me sirvió fue [fase], porque [razón].
>
> La diferencia con pedir código con un prompt suelto es que acá cada línea de código se puede rastrear hasta un escenario aprobado, y ese escenario hasta una decisión del Discovery.
>
> Y la lección más importante: el agente es muy capaz, pero completa lo que no sabe. Los puntos de control (Ajustar, revisar el spec, verificar fuentes) son lo que hace confiable el resultado.

Tiempos

Bloque	Duración
1. Discovery	1:15
2. Fundación de Active Stack	2:15
3. Explore, Propose y Ajustar	2:30
4. Apply y Verify	2:00
5. Archive	0:45
6. Engram	1:00
7. Conclusiones	1:00
Total	**10:45**

Si te pasás de 12 minutos, recortá primero las esperas del agente y después la demo.

Armado final en Clipchamp

1. **Crear un video nuevo** e importar los clips.
2. Ordenarlos en la línea de tiempo: 1 → 2 → 3 → 4 → 5 → 6 → 7. El arranque de la sesión de C-05 (Engram) va después del Archive.
3. Recortar con los bordes del clip o con **Dividir** (tijera). Las esperas del agente se cortan o se aceleran.
4. Verificar que el total quede entre 8 y 12 minutos.
5. **Exportar** → 1080p.